Member A — Data & Serving

You own: the synthetic data, then the API, the UI and Docker. **Hand this whole PDF to your Claude session and say:** "*Read this document and* `docs/DATA_SPEC.md`*, then work through Part 1 step by step. Explain each step to me in 2–3 plain lines before moving on.*"

> The **Shared Contract** is attached at the end of this PDF (also in `docs/team/SHARED_CONTRACT.md`). Your files: `scripts/generate_data.py`, `data/`, `src/api.py`, `ui/app.py`, `tests/test_api.py`, `Dockerfile`, `docker-compose.yml`, `requirements.txt`. Do not edit anyone else's files.**

Part 1 — Synthetic data (13:30 → 14:15, the whole team waits on this)

Follow `docs/DATA_SPEC.md` **exactly** — it is a complete brief. Order of work:

1. `scripts/generate_data.py`: hidden per-transformer attributes → monthly operation + failure process → oil tests → planted edge cases E1–E8 → late-drop rehearsal files → `data_card.json` + `DATA_CARD.md`.
2. Run it twice; confirm identical output (`certutil -hashfile data\operation.csv MD5`).
3. Check the printed summary: ~38% untested, 40–60 failures, ≥25 in months 0–23, faulty units fail clearly more.
4. **14:15 — ask your human to commit and push** `data/` **+ the script** so B and C can pull it. Suggested: `git add scripts/ data/` · `git commit -m "add synthetic data generator and data card"`.

Done when: all asserts in DATA_SPEC §9 pass and the files are pushed.

Part 2 — API with stubs (14:15 → 15:00)

Build `src/api.py` (FastAPI + Pydantic v2) **against the contract shapes first, with fake values**, so it works before B and C finish. Then swap in the real calls in Part 3.

- Endpoints (Contract §9): `POST /predict`, `POST /plan`, `POST /ingest`, `GET /model/info`, `GET /explain/{transformer_id}`, `GET /health`.
- Pydantic request models: `PredictRequest(transformer_ids: list[str] | None, as_of: str | None)`, `PlanRequest(budget: float, costs: dict | None)`.
- Response JSON exactly like HCL guide §6.1 — keys: `model_version`, `as_of`, `horizon_months` (12), `predictions[]` with `transformer_id, status, failure_prob_12m, prob_interval_90, health_score, impact_weight, risk_score, top_drivers, data_quality`. Every plan action carries `requires_approval: true`.
- `invalid_input` status: malformed body, unknown transformer id, or `as_of` earlier than the data.
- `/ingest`: multipart file + form field `table`; needs `python-multipart` (not installed — ask the human to run `pip install python-multipart` now, and add it to `requirements.txt`).
- `tests/test_api.py`: use FastAPI's `TestClient`; one test per endpoint checking status code + required keys.

Run locally: `uvicorn src.api:app --reload` → open `http://127.0.0.1:8000/docs`.

Part 3 — Wire it up, UI, Docker (15:00 → 15:40)

1. Replace stubs with real calls (Contract §9): `build_features` → `score` → `compute_risk`; `make_plan` + `age_plan`; `load_csv`; `model_info`. Write `fact_risk` / `fact_plan` rows via C's functions if they provide a helper, otherwise ask C.
2. `ui/app.py` (Streamlit, keep it plain — usability only is scored). Three pages via a sidebar radio:
   - **Risk ranking:** table of transformers sorted by `risk_score`, with status and a low-confidence marker.
   - **Transformer page:** pick an ID → probability + range, health score, top drivers, its oil-test history chart.
   - **Plan page:** budget input → chosen actions, total cost, risk removed, vs the age-based plan. The UI calls the API over HTTP (`API_URL` env var, default `http://api:8000` in Docker).
3. `requirements.txt` (pin versions from `pip show`): the exact list in Contract §2 rule 6 (pandas, numpy, sqlalchemy, scikit-learn, scipy, joblib, matplotlib, fastapi, uvicorn, pydantic, python-multipart, streamlit, requests, pytest, httpx). That is the list we gave the panel, so nothing more and nothing less.

4. `Dockerfile` (python:3.13-slim, `pip install -r requirements.txt`, copy repo) and `docker-compose.yml` with two services: `api` (`uvicorn src.api:app --host 0.0.0.0 --port 8000`, runs `python -m src.pipeline` first) and `ui` (`streamlit run ui/app.py --server.port 8501`). **Build the image early (by 15:15)** — `pip install` inside Docker needs internet, and venue wifi is slow.

Done when: `docker compose up` → `/health` is green, `/predict` returns real predictions, the UI shows them.

Part 4 — Demo rehearsal (15:40 → 16:10)

Rehearse the judges' live test (HCL §9.1): load `data/late_drop/oil_tests_late.csv` through `/ingest`, call `/predict` again, run `/plan` with budget 4,000,000, and show what changed. Write the sample `curl` commands for the README and send them to B.

What the others are building (so you know what to call)

- **B (Unique) — Architecture:** bronze → silver → gold in `warehouse.db`; `build_features()` (the shared feature function you call in `/predict`), `run_pipeline()`, `load_csv()` (your `/ingest`).
- **C — Model & Plan:** `score()`, `model_info()`, `compute_risk()`, `make_plan()`, `age_plan()`; `models/model.joblib` + `model_card.json`.

You must be able to explain (Q&A is individual)

How the data is generated and why it is realistic · each planted edge case · why every endpoint exists · how `/ingest` → re-score → re-plan works · what Docker compose starts.

Shared Contract — read by every team member and every Claude session

> This is the single source of truth that lets three people build in parallel. **If your code and this file disagree, this file wins.** If you need to change something here (a column, a function signature), **tell the other two humans first** — never change it silently.

---

1. The problem in 4 lines

A utility has ~400 transformers and money to fix only a few. We **predict each transformer's chance of failing in the next 12 months** (calibrated, with an uncertainty range), compute **risk = probability × impact** (hospital 5, industry 3, urban 2, rural 1, × customers served), and choose **replace / inspect / nothing** within a budget with a **knapsack**, beating an age-based plan. Served through FastAPI + Streamlit + Docker. Full HCL guide: `Problem_Statements/Transformer_Fleet_Health_and_Replacement_Planning_Participant_Guide_DS_ML.docx`.

Mentor answer (12:00, recorded): an inspection tells us how severe the transformer really is; if it is worse than the model predicted, its risk should go up. → See §8 for how that drives the plan.

---

2. Rules for every Claude session (non-negotiable)

1. **Never run a git command that writes** (`add`, `commit`, `push`, `pull`, `checkout`, …). Suggest it; the human runs it. No `Co-Authored-By` / tool trailers in suggested messages.
2. **Never write inside** `hackathon-kit/` (read-only reference). Root `CLAUDE.md` applies.
3. **The HCL guide overrides** `CLAUDE.md`**'s stack list.** Docker, FastAPI, scikit-learn model training and pytest tests are **required** by HCL — use them.
4. **Only edit the files your member owns** (§3). Need something from another member's file? Ask the human. This is what keeps `git pull` free of conflicts.
5. **Readable over clever.** Every line must be explainable by a student under questioning. Comment every non-obvious query and every ML step with one plain-English line.
6. **Libraries (this is the list we told the panel):** pandas, numpy, sqlalchemy, scikit-learn, scipy, joblib, matplotlib, fastapi, uvicorn, pydantic, python-multipart, streamlit, requests, pytest, httpx. **Nothing else** without asking. Only `streamlit` and `python-multipart` still need `pip install`.
7. **Python package style:** `src/__init__.py` exists; import as `from src.features import build_features`; run modules as `python -m src.pipeline` from the repo root.
8. **Seeds:** `SEED = 42` everywhere randomness appears.

---

3. Repo layout and ownership

Path	Owner	What
`scripts/generate_data.py`	**A**	synthetic data generator (spec: `docs/DATA_SPEC.md`)
`data/*.csv`, `data/late_drop/`, `data/data_card.json`, `data/DATA_CARD.md`	**A**	generated data + data card
`src/__init__.py`	**B**	empty file
`src/pipeline.py`	**B**	bronze → silver → gold, validation. `python -m src.pipeline`
`src/features.py`	**B**	the **shared feature function** (training *and* API use it) + labels
`src/loader.py`	**B**	CLI + function to load a judge CSV (`/ingest` calls it)
`src/model.py`	**C**	baselines, models, validation, calibration, intervals, `score()`
`src/plan.py`	**C**	risk, knapsack plan, age-based plan
`src/api.py`	**A**	FastAPI app — thin, calls B's and C's functions

`ui/app.py`	**A**	Streamlit UI (3 pages)
`tests/test_features.py`	**B**	leakage + validation tests
`tests/test_plan.py`, `tests/test_model.py`	**C**	budget never exceeded, one action per unit, …
`tests/test_api.py`	**A**	endpoints respond in the contract shape
`models/`	**C** (generated)	`model.joblib`, `model_card.json`
`reports/experiments.csv`, `reports/evaluation.md`	**C** (generated)	experiment log, evaluation report
`reports/validation_output.txt`	**B** (generated)	output of the validation step
`Dockerfile`, `docker-compose.yml`, `requirements.txt`	**A**	packaging
`README.md`	**B**	graded; A and C send their sections to B
`warehouse.db`	generated	SQLite warehouse — **gitignored**, rebuilt by the pipeline
`NOTES.local.md`	each person, own laptop	private, gitignored

4. Source files (produced by A, consumed by B)

Full detail in `docs/DATA_SPEC.md`. Summary: **Annex B columns are mandatory.** The *extra* ones are ours and may be missing in judges' files, so **B's pipeline must cope without them.**

File	Columns
`customers_served.csv`	`transformer_id, customer_type, customers, region` + *extra* `rating_class`
`operation.csv`	`transformer_id, month, load_pct, top_oil_temp_c, age_years` + *extra* `event` (`none` / `failure` / `repair`)
`oil_tests.csv`	`transformer_id, test_date, h2_ppm, ch4_ppm, c2h2_ppm, c2h4_ppm, c2h6_ppm, co_ppm`
`costs_budget.csv`	`action, rating_class, cost, annual_budget`

Data period: monthly 2023-10-01 → 2026-09-01. "Today" = 2026-10-11.

5. Warehouse tables (`warehouse.db`, SQLite) — built by B

Bronze — raw, never edited

Rebuilt on every pipeline run from **files on disk**: `data/<table>.csv` plus every file in `data/ingested/<table>__*.csv` (what `/ingest` saved). The files are the immutable replay tape.

Table	Columns
`bronze_customers_served`, `bronze_operation`, `bronze_oil_tests`, `bronze_costs_budget`	all source columns **as text, exactly as received** + `_ingested_at` (TEXT ISO) + `_source` (file name)

Silver — validated, cleaned, typed, one row per business key

Table	Key	Columns
`silver_transformer`	`transformer_id`	`customer_type, customers INT, region, rating_class, rating_class_assumed INT` (1 if extra column missing → `medium` used)
`silver_operation`	`transformer_id, month`	`load_pct REAL, top_oil_temp_c REAL, age_years REAL, event TEXT` (`'none'` if column absent)
`silver_oil_test`	`transformer_id, test_date`	`h2_ppm … co_ppm REAL` (all ppm), `unit_fixed INT` (1 if a ppb→ppm ×1000 jump was corrected)
`silver_costs`	`action, rating_class`	`cost REAL`
`silver_budget`	—	`annual_budget REAL` (one row; latest file wins)

Cleaning rules: IDs `strip().upper()`; exact duplicates dropped; duplicate keys → keep the latest `_ingested_at`; gaps stay **NULL** (never filled in silver).

Gold — star schema, shaped for reading

Table	Grain (one row = …)	Columns
`dim_transformer`	one transformer	`transformer_key INT` (surrogate), `transformer_id, customer_type, customers, type_weight INT, impact_weight REAL` (= type_weight × customers), `region, rating_class, rating_class_assumed`
`dim_date`	one calendar day	`date_key INT` (YYYYMMDD), `date, year, quarter, month, is_snapshot_date INT`
`fact_oil_test`	one transformer × one test	`transformer_key, date_key`, six gases, `tdcg` (sum of the six)
`fact_operation`	one transformer × one month	`transformer_key, date_key, load_pct, top_oil_temp_c, age_years, is_failure INT, is_repair INT`
`fact_snapshot`	**one transformer × one quarterly snapshot date**	`transformer_key, transformer_id, snapshot_date`, **all** `FEATURE_COLUMNS` **(§6)**, `failed_next_12m INT` (NULL = not labelable yet)
`dim_run`	one scoring / plan run	`run_id TEXT` (key), `created_at, kind ('score'/'plan'), as_of, budget, model_version`
`dim_action`	one action × size class	`action_key INT` (surrogate), `action ('replace'/'inspect'), rating_class, cost` — from `silver_costs`
`fact_risk`	one run × one transformer	`run_id, transformer_id, failure_prob_12m, prob_low, prob_high, status, health_score, impact_weight, risk_score, risk_low, risk_high, model_version`
`fact_plan`	one run × one chosen action	`run_id, transformer_id, action_key, action, cost, risk_before, risk_removed`

`dim_run`, `fact_risk`, `fact_plan` rows are **written by C's code** (via `src/plan.py` / `src/model.py`); B creates the empty tables and fills `dim_action`. Storing every run lets us show *what changed* after the late drop.

Shared (conformed) dimensions: one dimension serves several fact tables. This is what we told the panel:

Fact table ↓ / Dimension →	`dim_transformer`	`dim_date`	`dim_run`	`dim_action`
`fact_oil_test`	✓	✓ test date		
`fact_operation`	✓	✓ month		
`fact_snapshot`	✓	✓ snapshot date		
`fact_risk`	✓	✓ as-of date	✓	
`fact_plan`	✓	✓ as-of date	✓	✓

Customer type → weight (hospital 5, industry 3, urban 2, rural 1) is kept **inside** `dim_transformer` (`type_weight`) rather than in its own table: that is the star-schema choice (one join, a little repetition) over a snowflake.

Snapshot dates: first day of each quarter, **2024-01-01** → **2026-10-01**. **Labelable** only if `snapshot_date + 12 months ≤ 2026-10-01` → snapshots 2024-01-01 … 2025-10-01 have labels; later ones have `failed_next_12m = NULL` (used for scoring only). 2026-10-01 is "now".

---

6. The feature list — defined ONCE in `src/features.py` (owner B)

```
FEATURE_COLUMNS = [
    "age_years", "load_mean_12m", "load_p95_12m", "temp_mean_12m", "hot_months_12m",
    "has_oil_test", "n_tests", "months_since_last_test",
    "h2_last", "ch4_last", "c2h2_last", "c2h4_last", "c2h6_last", "co_last",
    "tdcg_last", "tdcg_delta", "c2h2_delta",
    "ratio_c2h2_c2h4", "ratio_ch4_h2", "ratio_c2h4_c2h6",
    "months_since_repair",
]
```

Every feature uses **only data dated strictly before** `snapshot_date` (this is the leakage rule).

Feature	Definition

`age_years`	from the latest `silver_operation` month before the snapshot
`load_mean_12m`, `load_p95_12m`	mean / 95th percentile of `load_pct` over months in [snapshot − 12 months, snapshot)
`temp_mean_12m`	mean `top_oil_temp_c`, same window
`hot_months_12m`	count of months in the window with `top_oil_temp_c > 80`
`has_oil_test`	1 if any usable test before the snapshot, else 0
`n_tests`	number of usable tests before the snapshot
`months_since_last_test`	(snapshot − last test date) in days / 30.44; NULL if no test
`*_last`	the six gases of the latest usable test; NULL if none
`tdcg_last`	sum of the six gases (total dissolved combustible gas) of the latest test
`tdcg_delta`, `c2h2_delta`	(last − previous) ÷ months between them; NULL if < 2 usable tests
`ratio_c2h2_c2h4`, `ratio_ch4_h2`, `ratio_c2h4_c2h6`	gas ratios of the latest test, denominator + 0.1 to avoid ÷0; NULL if no test
`months_since_repair`	months since the latest `repair` **or** `failure` event before the snapshot; NULL if none

"Usable test" = a test **after** the most recent repair/failure event before the snapshot (a repair resets the gas pattern — edge case E5). **Missing stays NULL** — handling missing values is the model's job (C).

Label: `failed_next_12m = 1` if `silver_operation` has `event = 'failure'` in a month **m** with `snapshot_date ≤ m < snapshot_date + 12 months`; `0` if not; `NULL` if not labelable (§5).

7. Function interfaces — the hand-off points

```
# src/features.py  (B)
FEATURE_COLUMNS: list[str]
def build_features(conn, snapshot_dates: list[str]) -> pd.DataFrame:
    """Columns: transformer_id, snapshot_date, *FEATURE_COLUMNS. Reads silver tables only."""
def build_labels(conn, snapshot_dates: list[str]) -> pd.DataFrame:
    """Columns: transformer_id, snapshot_date, failed_next_12m."""

# src/pipeline.py  (B)
DB_PATH = "warehouse.db"
def get_connection(): ...                    # sqlite3 / SQLAlchemy connection to DB_PATH
def run_pipeline() -> dict:                  # rebuild bronze → silver → gold; returns row counts per table

# src/loader.py  (B)
def load_csv(table: str, csv_path: str) -> dict:
    """table ∈ {oil_tests, operation, customers_served, costs_budget}.
    Checks required Annex B columns, copies the file to data/ingested/<table>__<timestamp>.csv,
    re-runs the pipeline. Returns {"rows_loaded": int, "rows_rejected": int, "status": "ok"|"error", "detail":
str}."""

# src/model.py  (C)
MODEL_PATH = "models/model.joblib"; CARD_PATH = "models/model_card.json"
def train() -> dict:                         # reads fact_snapshot, runs the full evaluation, saves model +
card + reports
def score(features: pd.DataFrame) -> pd.DataFrame:
    """In: output of build_features. Out per transformer: transformer_id, failure_prob_12m, prob_low, prob_high,
    status, health_score, top_drivers (list of {"feature","contribution"}), data_quality, model_version."""
def model_info() -> dict:                    # contents of model_card.json

# src/plan.py  (C)
TYPE_WEIGHTS = {"hospital": 5, "industry": 3, "urban": 2, "rural": 1}
def compute_risk(scores: pd.DataFrame, dim_transformer: pd.DataFrame) -> pd.DataFrame:  # adds impact_weight,
risk_score, risk_low, risk_high
def make_plan(risk: pd.DataFrame, costs: pd.DataFrame, budget: float) -> dict:
    """{"actions": [{"transformer_id","action","cost","risk_before","risk_removed","rationale"}],
        "total_cost", "budget", "risk_before", "risk_removed", "risk_reduced_pct"}"""
def age_plan(risk: pd.DataFrame, costs: pd.DataFrame, budget: float) -> dict:          # same shape, replaces
oldest first
```

Status values (HCL §6.2): `scored` · `scored_low_confidence` (no oil test, last test > 18 months old, or only 1 test) · `insufficient_data` (no operation rows) · `invalid_input` (API only) · `plan` (API `/plan` only).

---

8. Decision logic — how replace vs inspect is decided (owner C)

1. **Model** gives each transformer `p` (calibrated) and a range `[p_low, p_high]`. Few/old/no tests → wide range.
2. **Value of each action** (risk removed):
   - **Replace:** `(p − 0.01) × impact`. A replacement removes all the risk except a new unit's own, which we assume is a 1% yearly failure chance.
   - **Inspect:** `(p_high − p) × impact`. An inspection removes a **stated fraction** of the risk, and that fraction is **how uncertain we are about the unit**: the hidden risk an inspection could reveal. Wide range (we know little) → inspection is valuable. Narrow range (we already know) → it is worth about 0. *Why not one fixed fraction (e.g. 30%)?* Inspection costs ~1/23 of a replacement, so with a fixed fraction the optimiser inspects almost every unit and replaces almost none. Tying the fraction to uncertainty sends inspections to where they actually tell us something.
   - **Rule:** a transformer with **no usable oil test or a last test > 18 months old cannot be replaced blind** — only inspected (don't spend ~350k on something we have not looked at).
3. **Knapsack** picks at most one action per transformer, maximising total risk removed, total cost ≤ budget (dynamic programming over cost in units of 5,000 — all costs are multiples of 5,000).
4. **Inspection feedback loop (mentor's point):** an inspection *is* a new oil test. Its result arrives through `/ingest` (`oil_tests`) → pipeline rebuilds → model re-scores → if severe, `p` goes **up** and the range **narrows** → replace value rises, inspect value falls → **the next plan replaces it** (if budget allows). If the test is clean, `p` goes down and it drops off the list. The late-drop file (new oil tests) demonstrates exactly this. **A human engineer approves every plan** (`requires_approval: true`).

8b. Assumptions the panel told us to make (12:00 design review)

Costs and budget are **ours to assume**. We assume these values and write them into the README "Assumptions" section:

Action	small	medium	large
replace	150,000	350,000	700,000
inspect	10,000	15,000	25,000

Annual budget **5,000,000**; late-drop rehearsal budget **4,000,000** (a 20% cut). This is already in `costs_budget.csv` (DATA_SPEC §3.4).

8c. Health index: a score for each field (owner C, in `src/model.py`)

We told the panel each field gets its own score. This is that rule. It is used three ways: (1) as the `health_score` returned by the API, (2) as the **rule-based baseline** the ML model must beat, and (3) as an **explanation** ("gas score 30 because acetylene is at condition 3"). Gas limits follow the **IEEE C57.104 dissolved-gas "condition" levels** (C1 normal → C4 severe). **Verify the numbers against the standard before quoting them.**

Field	Condition 1 (normal)	C2	C3	C4 (severe)
H2 (ppm)	≤ 100	101–700	701–1800	> 1800
CH4	≤ 120	121–400	401–1000	> 1000
C2H2	≤ 1	2–9	10–35	> 35
C2H4	≤ 50	51–100	101–200	> 200
C2H6	≤ 65	66–100	101–150	> 150
CO	≤ 350	351–570	571–1400	> 1400

Score component	Points deducted from 100
Gas score: worst gas condition	C1 → 0 · C2 → 15 · C3 → 30 · C4 → 40
Trend score: `tdcg_delta` (ppm per month)	≤ 10 → 0 · 10–30 → 10 · > 30 → 20
Load score: `load_p95_12m`	≤ 90 → 0 · 90–100 → 10 · > 100 → 20
Age score: `age_years`	≤ 20 → 0 · 20–30 → 6 · 30–40 → 12 · > 40 → 20

`health_score = 100 − (gas + trend + load + age)`, clipped to 0–100. **NULL when there is no usable oil test**, because the gas and trend scores cannot be computed, and an untested unit must never look healthy. The model's *probability* is still produced for such units (from age, load and `has_oil_test`), flagged low confidence.

---

9. API (owner A) — which function each endpoint calls

Endpoint	Calls
`POST /predict` (optional `transformer_ids`, `as_of`)	`build_features(conn, [as_of or "2026-10-01"])` → `score()` → `compute_risk()`; writes `fact_risk`
`POST /plan` (`budget`, optional costs)	latest `fact_risk` → `make_plan()` + `age_plan()`; writes `fact_plan`
`POST /ingest` (multipart CSV + `{"table": ...}`)	`load_csv()`
`GET /model/info`	`model_info()`
`GET /explain/{transformer_id}`	`score()` for that unit → `top_drivers` + its latest tests/operation rows
`GET /health`	DB reachable? model file loaded?

Response shapes: HCL guide §6.1 (`predictions[]` with `failure_prob_12m`, `prob_interval_90`, `health_score`, `impact_weight`, `risk_score`, `top_drivers`, `data_quality`, `status`, `model_version`).

---

10. Timeline and sync points (code freeze 16:15)

Time	A — Data & Serving	B — Architecture (Unique)	C — Model & Plan
13:15–13:30	read docs, set up	read docs, set up	read docs, set up

13:30–14:15	**generator + data card**	bronze + silver on a 5-row hand-made sample	`plan.py` + its tests (toy data)
14:15 sync	**push** `data/`	pull data, run on real files	—
14:15–15:00	`api.py` with stub responses in contract shape	gold + `features.py` + labels + leakage test	`model.py` on a stand-in `fact_snapshot`
15:00 sync	—	**push** `fact_snapshot` **code**	pull, train on real data
15:00–15:40	wire API to real functions; Streamlit; Dockerfile	`loader.py`, validation output, README	evaluation, calibration, model card, experiment log
15:40 sync	`docker compose up` works	README sections merged	**push** `models/` **+** `reports/`
15:40–16:10	rehearse demo end-to-end, fix, tag `final`		

Cut list if behind (in this order): Streamlit to one page → bootstrap intervals from 20 not 50 models → drop the `/explain` sensor detail. **Never cut:** time-based grouped validation, calibration, the knapsack, the API.

---

11. Git workflow (the humans run these, not Claude)

- Everyone works on `main`, touching only their own files (§3).
- Before pushing: `git pull --rebase` then `git push`. Commit **at least every 30–45 minutes**, small messages.
- Commit `data/` and `models/` (judges and Docker need them). Never commit `warehouse.db`, `.env`, `NOTES.local.md`.

Synthetic Data Spec — Transformer Fleet Health (HCL DS/ML edition)

> **Read this whole file before writing any code.** It is a complete, self-contained brief for generating the four CSV files the team needs. HCL does **not** provide data — we generate all of it. If anything here is unclear or impossible, **stop and ask the human** rather than guessing.

---

0. Ground rules for the agent doing this

- **Never run a git command that writes** (`add`, `commit`, `push`, etc.). Suggest the command; the human runs it. No `Co-Authored-By` or tool trailers in suggested commit messages. (See `CLAUDE.md`.)
- **Never write inside** `hackathon-kit/` — it is read-only reference.
- **Libraries: Python stdlib + `numpy` + `pandas` only.** Nothing else is needed. Do not pip install.
- **Deterministic:** one fixed seed (`SEED = 42`) via `numpy.random.default_rng(SEED)`. Running the script twice must produce byte-identical CSVs.
- **Readable code** — the human must explain every line to a judging panel. One file, clear functions, a comment on anything non-obvious. No classes, no config systems, no clever one-liners.
- **No real company or personal data** anywhere. IDs and regions are invented.

What to produce

Path	What
`scripts/generate_data.py`	The generator. `python scripts/generate_data.py` writes everything below.
`data/customers_served.csv`	Asset master (one row per transformer, plus planted duplicates)
`data/operation.csv`	Monthly loading / temperature / age, plus failure & repair events
`data/oil_tests.csv`	Sparse dissolved-gas test results
`data/costs_budget.csv`	Action costs per size class + annual budget
`data/late_drop/oil_tests_late.csv`	Rehearsal copy of the 13:45 late data drop (new tests)
`data/late_drop/costs_budget_late.csv`	Rehearsal copy of the late drop (budget cut 20%)
`data/data_card.json`	Machine-readable: row counts per file + IDs of every planted edge case
`data/DATA_CARD.md`	Human-readable data card (HCL Annex D — template in §8)

---

1. Why this data exists (context the generator must respect)

The team builds an ML model that predicts **the chance each transformer fails in the next 12 months**, then picks which to replace/inspect within a budget. **On judging day HCL loads *their own* hidden data (same schema) through our loader and scores our model against real outcomes.** So:

1. **Values must be physically realistic** (typical dissolved-gas levels, loading, temperatures). A model trained on silly data will score badly on theirs.
2. **Failures must be learnable but noisy** — driven by age, loading and developing gas faults, with randomness. Not a perfect rule, not pure noise.
3. **Every edge case HCL lists must be planted on purpose** and its IDs recorded in the data card, so the team can show each trap being handled in the demo.

---

2. Global settings

Setting	Value
Seed	`42`
Number of transformers	**400** — IDs `TR-0001` … `TR-0400` (4-digit zero-padded)
Operation period	**36 months: 2023-10-01 to 2026-09-01** (first day of each month)
Oil-test dates	between 2023-10-01 and 2026-09-30

Late-drop test dates	between 2026-10-01 and 2026-10-10
"Today"	2026-10-11 — **no date anywhere may be later than this**
CSV format	UTF-8, comma-separated, header row, **no index column**, `.` decimal, dates `YYYY-MM-DD`
Missing values	empty cell (pandas writes NaN as empty — that is correct)

3. The four files — exact schema

Annex B columns are mandatory and must be spelled exactly as below. We add a few **extra columns** (Annex B allows extras) — marked *EXTRA*. The team's pipeline must still work without them, because the judges' files will only contain Annex B columns.

3.1 `customers_served.csv` — the asset master

Column	Type	Rule
`transformer_id`	string	`TR-0001` … `TR-0400`
`customer_type`	string	`hospital` 10% · `industry` 20% · `urban` 35% · `rural` 35%
`customers`	int	by type — hospital 5,000–20,000 · industry 50–2,000 · urban 2,000–15,000 · rural 200–3,000 (uniform int)
`region`	string	one of `North`, `South`, `East`, `West`, `Central` (uniform)
`rating_class` *EXTRA*	string	`small` 50% · `medium` 35% · `large` 15% — needed to price actions

Rows: 400 + planted duplicates (§6). **This file is the "asset list"** referred to in HCL's validation checks.

3.2 `operation.csv` — monthly operation

Column	Type	Rule
`transformer_id`	string	
`month`	date	first of month, 2023-10-01 … 2026-09-01
`load_pct`	float	average load as % of rating, 1 decimal (see §4.2)
`top_oil_temp_c`	float	°C, 1 decimal (see §4.2)
`age_years`	float	age at that month, 2 decimals; increases by 1/12 each month
`event` *EXTRA*	string	`none`, `failure` or `repair` — the **training labels** come from this

Rows: 400 × 36 = 14,400, minus dropped gap rows, plus planted duplicate rows (§6).

3.3 `oil_tests.csv` — dissolved gas analysis (sparse)

Column	Type	Rule
`transformer_id`	string	only transformers that are "tested" (§4.3)
`test_date`	date	
`h2_ppm`	float	Hydrogen
`ch4_ppm`	float	Methane
`c2h2_ppm`	float	Acetylene
`c2h4_ppm`	float	Ethylene
`c2h6_ppm`	float	Ethane
`co_ppm`	float	Carbon monoxide

All gases: **strictly positive** (floor at 0.5), 1 decimal, unit **ppm** (except the planted unit-jump rows, §6).

3.4 `costs_budget.csv` — small reference table (exactly 6 rows)

action	rating_class	cost	annual_budget
replace	small	150000	5000000

replace	medium	350000	5000000
replace	large	700000	5000000
inspect	small	10000	5000000
inspect	medium	15000	5000000
inspect	large	25000	5000000

(`annual_budget` repeats on every row — that is how Annex B lays it out.) **These costs and the budget are our assumptions. At the 12:00 design review the panel told us to assume them.** Record that in `DATA_CARD.md`.

---

4. How to generate it — the hidden "true" story

Generate in this order. Keep the hidden per-transformer attributes in a pandas DataFrame (`assets`) while generating; **do not write hidden attributes to the CSVs** except where stated (they go into `data_card.json` for the team's own checking).

4.1 Per-transformer hidden attributes

Attribute	Rule
`age_at_start` (on 2023-10-01)	25% uniform 2–10 · 40% uniform 10–30 · 35% uniform 30–50 years
`base_load`	normal(65, 15), clipped to 20–120. **Plus 10 overloaded units** with base_load uniform 95–115
`load_trend`	20% of units grow +0.2 load-points per month; others 0
`is_tested`	has any oil test. **Target ~62% tested, ~38% untested** (HCL minimum: ≥30% untested). Make rural and small units **less likely** to be tested (e.g. tested-probability 0.50 for rural/small, 0.70 otherwise, then adjust to hit ~62%)
`fault`	**40 units** get a developing fault (~10%). Type: `thermal` 50% · `partial_discharge` 30% · `arcing` 20%. **At least 25 of them must be tested units with ≥2 tests after onset** (HCL minimum: ≥10 clear developing faults visible in the data)
`fault_onset_month`	uniform integer 0–30 (index into the 36 months)

4.2 Monthly operation (for every transformer, every month m = 0…35)

- **Season** (Northern hemisphere, hot May–Aug): `season = sin(2π · (calendar_month − 2) / 12)`
- `load_pct = base_load + load_trend·m + 8·season + normal(0, 4)`, clipped 5–150
- `ambient = 25 + 10·season`
- `top_oil_temp_c = ambient + 0.45·load_pct + normal(0, 2)`, **+5 if a thermal fault is active**, clipped 15–120
- `age_years = age_at_start + m/12`

4.3 Failure process (this creates the labels — get it right)

For each transformer, walk month by month and draw a failure with monthly probability `h`:

```
h = base · age_mult · load_mult · fault_mult · untested_mult
base          = 0.0012                          # tune, see below
age_mult      = exp((age_years − 25) / 15)      # 50-year-old ≈ 5×, 10-year-old ≈ 0.4×
load_mult     = 1 + max(0, load_pct − 85) / 20 # overload raises risk
fault_mult    = 1 if no active fault, else 1 + 0.6 · min(months_since_onset, 15)
untested_mult = 1.4 if not is_tested else 1    # PLANTED: neglected units fail more — disclose it
```

- If a failure is drawn: set that month's `event = "failure"`. The unit is **repaired the same

month**: its active fault is cleared and its gases return to baseline from the next test on. It stays in the data under the same ID (it can, rarely, fail again).

- **Tune** `base` so total failures over 36 months land in **40–60** (~3–4% per year — realistic and

rare). Do it deterministically: run once, then rescale `base` by `50 / actual` and regenerate once. Print the final count. **Assert** at least 25 failures land in months 0–23 (the team needs 12 months of future to label a snapshot, so only the first 24 months are labelable).

- **Assert** that units with an active fault have a clearly higher failure rate than units without

(the signal must be learnable). Print both rates.

4.4 Oil tests (tested units only)

- First test in month uniform 0–8; then every 6–12 months (uniform int) until month 35 →

typically 3–6 tests per unit. `test_date` = a random day within that month.

- **Baseline gases** per transformer (draw once, lognormal around these medians, σ = 0.3):

Gas	Baseline median (ppm)	Notes
h2	40	
ch4	30	
c2h2	0.5	80% of units sit at the 0.5 floor; 20% at 1–2
c2h4	15	
c2h6	20	
co	150 + 6 · age_years	CO rises with paper insulation ageing

- **Developing fault growth**, `k = months since onset` while the fault is active:

Fault	Gas multipliers on baseline
`thermal`	ch4 × (1 + 0.12k) · c2h4 × (1 + 0.25k) · c2h6 × (1 + 0.08k)
`partial_discharge`	h2 × (1 + 0.30k) · ch4 × (1 + 0.05k)
`arcing`	c2h2 + 2.0·k (additive) · h2 × (1 + 0.20k) · c2h4 × (1 + 0.10k)

- Each test value = (baseline × multipliers) × lognormal noise (σ = 0.2), floored at 0.5, rounded to 1 decimal.

These ratios are deliberately the textbook patterns (thermal → ethylene/methane, partial discharge → hydrogen, arcing → acetylene), so a documented gas rule (IEEE C57.104 / Duval-style) works on them.

---

5. HCL minimums — the generator must assert all of these

- [ ] ≥ 300 transformers (we use 400)
- [ ] ≥ 30% of transformers have **no** oil test
- [ ] ≥ 10 transformers with a **clear developing fault** visible in their tests (≥2 rising tests after onset)
- [ ] ≥ 2 look-alike pairs serving very different customers (we plant 3, §6)

---

6. Planted edge cases (HCL §4.4) — exact recipe, record every ID in `data_card.json`

#	Edge case	How to plant it
E1	**No oil test at all**	the ~38% untested units (§4.1)
E2	**Old transformers with few measurements**	≥ 6 tested units with `age_at_start` > 40 that get **exactly 1** oil test
E3	**Look-alike pairs, different impact**	3 pairs. Second unit of each pair copies the first's `age_at_start`, `base_load`, `load_trend`, `rating_class`, `fault` and test schedule; its gas values = first unit's × lognormal(σ 0.05). One unit is `hospital` with 8,000–15,000 customers, the other `rural` with 200–600
E4	**Gas values jump because of a changed unit**	3 tested units (≥3 tests each): pick one **non-first** test and multiply **all six gases by 1000** (recorded in ppb under the ppm column). No unit column — the jump must be detectable from the data
E5	**Recent repair resets the pattern**	3 tested units with a developing fault: in a month 2–4 months before 2026-09 set `event = "repair"`, clear the fault from then on (gases back to baseline at later tests). Make sure each has ≥1 test before and ≥1 test after the repair
E6	**Budget cannot cover all high-risk units**	already true by design: 40 faulty units × replace cost ≫ 5,000,000. **Assert** total replace cost of all faulty units > `annual_budget`
E7	**Duplicated assets**	`customers_served.csv`: 3 exact duplicate rows, plus 2 near-duplicates of existing IDs with messy formatting (`"tr-0045"` lowercase, `"TR-0046 "` trailing space). `operation.csv`: for 2 units, duplicate 3 month-rows exactly

E8	**Gaps**	drop ~2% of `operation.csv` rows at random (missing months); set ~1% of `load_pct` and ~1% of `top_oil_temp_c` to empty

Make sure planted units do not collide in a way that breaks a recipe (e.g. an E2 unit should not also be an E4 unit). Pick them with the seeded RNG from eligible units and record them.

---

7. Late-drop rehearsal files (`data/late_drop/`)

HCL drops a late file at 13:45: "*new oil test results for some transformers, and a changed budget.*" Generate a rehearsal version so the team can practise:

- `oil_tests_late.csv` — same schema as `oil_tests.csv`, dates 2026-10-01 … 2026-10-10:

new tests for 30 already-tested units (continue their gas story) **plus 10 previously-untested units** getting their first-ever test (at least 2 of these 10 should show a fault pattern).

- `costs_budget_late.csv` — same 6 rows, `annual_budget = 4000000` (a 20% cut).

---

8. Data card — `data/data_card.json` and `data/DATA_CARD.md`

`data_card.json` — the team's validation script reads this to check "row counts match the data card":

```
{
  "name": "transformer_fleet_synthetic", "version": "1.0", "created": "2026-10-11", "seed": 42,
  "row_counts": {"customers_served.csv": 0, "operation.csv": 0, "oil_tests.csv": 0, "costs_budget.csv": 6},
  "n_transformers": 400, "n_untested": 0, "n_failures": 0, "n_failures_months_0_23": 0,
  "edge_cases": {
    "E2_old_few_tests": [], "E3_lookalike_pairs": [["TR-....", "TR-...."]],
    "E4_unit_jump": [{"transformer_id": "TR-....", "test_date": "YYYY-MM-DD"}],
    "E5_repair_reset": [{"transformer_id": "TR-....", "repair_month": "YYYY-MM-01"}],
    "E7_duplicate_assets": [], "E7_duplicate_operation": []
  },
  "faulty_units": [{"transformer_id": "TR-....", "fault": "thermal", "onset_month": "YYYY-MM-01"}]
}
```

(fill real values; `faulty_units` is our own answer key — the **model must never read it**; it is only for the team's sanity checks and the demo narrative.)

`DATA_CARD.md` — HCL Annex D fields, in plain English:

Field	What to write
Name and version	`transformer_fleet_synthetic` v1.0, 2026-10-11
How it was made	Script `scripts/generate_data.py`, seed 42, numpy + pandas. **Paste the prompt(s) given to the AI assistant** and name the model used (HCL requires this)
What it contains	each file, row count, date range
Traps and edge cases	E1–E8 with the IDs and what each one tests
Checks run	the asserts from §5 and §9 and their results
Known limits	failure process is a simple hazard formula; gases follow textbook patterns only; no weather, no repair costs, untested units planted to fail 1.4× more often
Privacy	confirm no real company or personal data

---

9. Final self-checks — the script must `assert` these and print a summary

1. Every `transformer_id` in `operation.csv` and `oil_tests.csv` exists in `customers_served.csv` (after stripping/uppercasing the messy duplicates)
2. All gas values > 0
3. No date later than 2026-10-11; all main oil-test dates ≤ 2026-09-30
4. Every transformer has a `customer_type`

5. All costs and budgets > 0
6. All §5 minimums hold
7. Failure total in 40–60; ≥25 in months 0–23; faulty units fail at a clearly higher rate
8. Row counts written to `data_card.json` equal the actual CSV row counts
9. Running the script twice gives identical files (the human can check with `certutil -hashfile data\operation.csv MD5` on Windows)

The summary printed at the end should look like:

```
customers_served.csv : 405 rows (400 unique + 5 planted duplicates)
operation.csv        : 14,1xx rows
oil_tests.csv        : ~1,000 rows, 248 tested units, 152 untested (38%)
failures             : 52 total, 34 in months 0-23
fault vs no-fault monthly failure rate: 0.0xx vs 0.00x
edge cases planted   : E2=6  E3=3 pairs  E4=3  E5=3  E7=5+2
All checks passed.
```

---

10. When done — hand back to the human

Tell the human, in 5–6 plain lines: what the script does, the final counts, where the edge cases are, and anything you were unsure about. Then suggest (do not run):

```
git add scripts/generate_data.py data/ docs/DATA_SPEC.md
git commit -m "add synthetic data generator and data card"
```